Creative Software Design
4. Lab2
C++ References

Dept. of Computer Science

Hanyang University

[IMPORTANT] Submission Guide

- Pull to your project directory before each lab session
  - At your VM instance

```
$ cd Projects/{your directory}
$ git pull origin main
$ cd week04_2
```

C++ References

- References can be used similar to pointers
  - Internal ➔ Pointer variables
  - Look ➔ Non-pointer variables

- Less powerful but safer than the pointer type
  - Less powerful ➔ Reassigning a new address is not allowed
  - Safer ➔ References always have valid address

```
int a = 10;

int* pa = &a;
*pa = 15;
```

Pointers

```
int a = 10;

int& ra = a;
ra = 15;
```

References

#1 – What C++ References Store?

- A Reference variable looks same as a non-pointer variable
- But it actually stores the address of the referenced variable
- **gdb** helps you to examine local variables

  info local or i local

```
#include <iostream>

int main(void) {

  int a = 10;
  int* pa = &a;
  int& ra = a;

  ra = a + 10;
  std::cout << a << " @" << &a << std::endl;
  std::cout << *pa << " @" << pa << std::endl;
  std::cout << ra << " @" << &ra << std::endl;

  return 0;
}
```

```
(gdb) i local
a = 20
pa = 0x7fffffffe364
ra = @0x7fffffffe364: 20
(gdb)
```

#2 – Reference to Array

- References can be used to refer to arrays
  - ~~A reference to a pointer is also possible!!~~

```
#include <iostream>

using namespace std;

int main(void) {
  char txt[10];
  // ref is a reference to txt //

  cin >> txt;
  cout << "ref: " << ref << ", len: " << sizeof(ref) << endl;
  return 0;
}
```

```
$ ./example02
Hello
ref: Hello, len: 10
```


#3 – swap()

- Complete two versions of the swap function
  - One for pointer variables, the other for references

```
#include <iostream>

using namespace std;

/* implement here */

int main(void) {
  int a, b;

  a = 10, b = 20;
  swap_by_pointers(&a, &b);
  cout << "a: " << a << ", b: " << b << endl;

  a = 10, b = 20;
  swap_by_references(a, b);
  cout << "a: " << a << ", b: " << b << endl;
  return 0;
}
```

```
$ ./example03
a: 20, b: 10
a: 20, b: 10
```

Out-parameters?

- Three different types of returning a value from a function

```
#include <iostream>

int add0(int x, int y) {
  return x + y;
}

void add1(int x, int y, int* ret) {
  *ret = x + y;
}

void add2(int x, int y, int& ret) {
  ret = x + y;
}

int main(void) {
  ret val0, val1, val2;
  val0 = add0(10, 20);
  add1(10, 20, &val1);
  add2(10, 20, val);
  return 0;
}
```


#4 – Center of Points

- Complete the function that calculates the center of the given points
  - All points are given as an array
  - The result should be provided by a reference
    - Either a return value or an out-parameter

    What if the coordinates of the center are not integers?

```
struct _point {
    int x;
    int y;
};

struct _point points[4] = {{8, 3}, {7, 5}, {5, 1}, {4, 9}};
```

All Done!!

- Let's submit to ***gitlab***
- Push to ***gitlab*** after each lab session
  - At your VM instance (and make sure **pwd** at "week04_2")

```
# You are at {Your local project directory}/week04_2
$ git add *.cpp
$ git commit –m "submission of week04_2"
$ git push origin main
```